

I design and implement complex software systems in C++ and .NET, from compiler/runtime infrastructure and program analysis to backend/service systems. I am especially interested in architecture-heavy problems where component boundaries, correctness, recovery, and verifiable invariants matter.

2022: Compilers / VMs · 2023: CIL + native x86-64 · 2024–2025: AST / IR / backends → toolchain · 2026 → : LLVM + program analysis

Systems engineering

C++/.NET across compiler/runtime infrastructure, program analysis, backend/service systems, and low-level codegen.

Architecture ownership

UniversalToolchain: one semantic planner and immutable plan; VpnMediator: explicit state/recovery; x86-64 backend: independent verifier plus differential execution.

Technical depth

MCST: professional LLVM 22/C++23 internship · ISP RAS selection task: Roslyn/C#/.NET static analysis.

EXPERIENCE

July — August 2026

MCST — Compiler Engineering Intern

- Implemented a LICM pass for LLVM 22 in C++; hoisting conditions check loop structure, memory access, side effects, and speculative safety.
- Built a Python harness comparing program behavior before/after optimization and checking transform/no-transform cases.

PROJECTS

UniversalToolchain / Wist2

One public semantic planner resolves dependencies/conflicts, selects providers, and freezes contribution ordering plus typed backend routes into one immutable LanguagePlan before runtime starts.

The design is enforced by pre-execution validation, exact manifest/ownership checks, and interpreter/CIL parity; the current exact manifest is 1,324/1,324.

VpnMediator

Explicit payment/access states separate external-event acceptance from idempotent reconciliation and recovery; migrations and operational gates keep lifecycle transitions inspectable.

Recovery includes backup/restore, health-gated release switching, and rollback; the public case study excludes secrets and user data.

x86-64 Compiler Backend & Register Allocation

Register-allocation strategy is separated from an independent assignment verifier before phi lowering and SysV x86-64 emission.

Generated native behavior is differentially checked against a reference interpreter; tests cover branches, loops, spills, and parallel phi moves.

SKILLS

Software / Systems

C++23, C#/.NET, Python, Linux, backend/runtime infrastructure, low-level/system components

Architecture

ownership/module boundaries, typed contracts, immutable planning, explicit state machines, lifecycle/recovery

Compiler / Program Analysis

LLVM 22/IR, CFG/SSA, .NET/CIL, static/data-flow analysis, optimization legality

Reliability / Verification

idempotency, reconciliation, backup/restore, regression tests, independent verifiers, differential execution

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

HSE Faculty of Computer Science — open-source projects competition

Selected as one of the winners with UniversalToolchain / Wist2.

LangDev’26 — Build the Language, Then Make the Abstractions Disappear

Accepted talk on UniversalToolchain’s architecture.

Moscow / remote